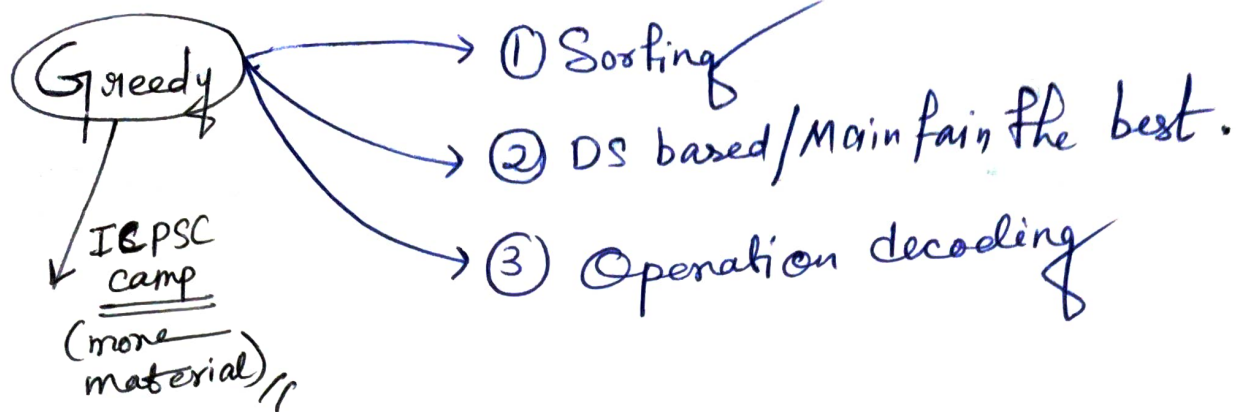




Q1 $A = [2, 3, 4, 5, 1]$
 $B = [-1, 3, 9, -2, 4]$

$$-10^5 \leq A(), B() \leq 10^5$$

- ① Rearranging allowed $\rightarrow (A, B)$
- ② $\text{Min}(\langle A \cdot B \rangle)$
 dot product.

Solⁿ

$$\begin{matrix} [a, b, c] \\ [x, y, z] \end{matrix} \rightarrow (a \cdot x + b \cdot y + c \cdot z) \downarrow$$

$$\begin{matrix} \swarrow & \searrow & \searrow \\ (ASC, ASC) & (ASC, DSC) & (DSC, DSC) \equiv (ASC, ASC) \end{matrix}$$

Sample Test Case

$$\begin{matrix} [1, 2, 3] \\ [-1, 0, 1] \end{matrix} \xrightarrow{ASC, ASC} \begin{matrix} 1, 2, 3 \\ -1, 0, 1 \end{matrix} \Rightarrow -1 + 0 + 3 = +2$$

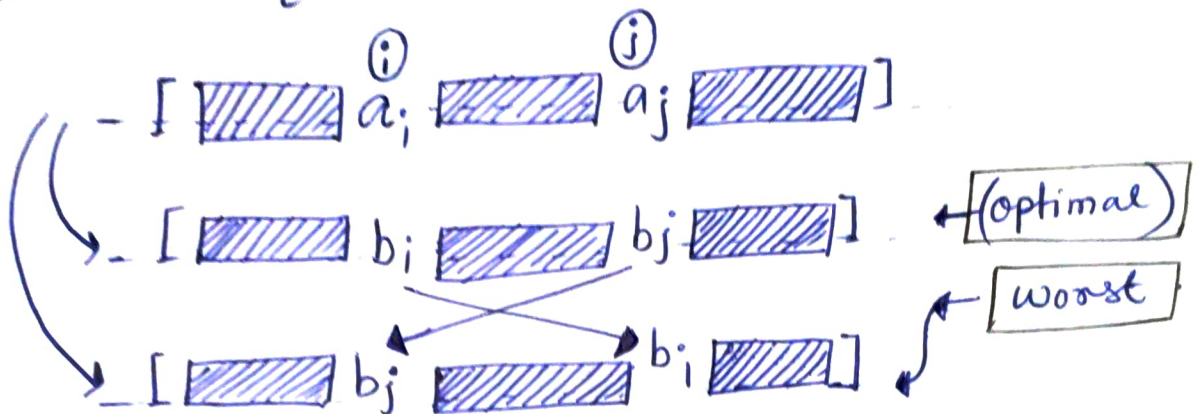
$$\begin{matrix} [1, 2, 3] \\ [-1, 0, 1] \end{matrix} \xrightarrow{ASC, DSC} \begin{matrix} 1, 2, 3 \\ 1, 0, -1 \end{matrix} \Rightarrow 1 + 0 - 3 = -2 \text{ minimum}$$

So, Probably (Asc, Dsc) is my answer. But qstn is
How to be sure that my intuition is correct??

Swapping proof

$$A \rightarrow [a_1 \leq a_2 \leq a_3 \leq a_4 \dots \leq a_n]$$

$$(\text{optimal}) \leftarrow B \rightarrow [b_1 \ b_2 \ b_3 \ b_4 \dots b_n]$$



$$\langle A_{\text{OPT}} \rangle \leq \langle A_{\text{Worst}} \rangle$$

$$\left(\begin{array}{l} \cancel{a_1 b_1} + \cancel{a_2 b_2} + \dots \\ \dots + a_i b_j + \dots \\ \dots + a_j b_i + \dots \\ \dots + \cancel{a_{n-1} b_{n-1}} + \cancel{a_n b_n} \end{array} \right) \leq \left(\begin{array}{l} \cancel{a_1 b_1} + \cancel{a_2 b_2} + \dots \\ \dots + a_i b_j + \dots \\ \dots + a_j b_i + \dots \\ \dots + \cancel{a_{n-1} b_{n-1}} + \cancel{a_n b_n} \end{array} \right)$$

chunk of element common in all the three arrays

$$a_i b_i + a_j b_j \leq a_i b_j + a_j b_i$$

$$a_i b_i + a_j b_j - a_i b_j - a_j b_i \leq 0$$

$$a_i (b_i - b_j) - a_j (b_i - b_j) \leq 0$$

$$\underbrace{(a_i - a_j)}_{\text{Since } a_j > a_i} \underbrace{(b_i - b_j)}_{\text{this needs to be } \oplus \text{ve.}} \leq 0$$

So $(a_i - a_j) < 0$

So, hence proved
that array B must
be sorted in decreasing
order) $\leftarrow (b_i > b_j)$

Code in Github link!

Q2 N-problems, are given with their respective decay and time required by the coder to solve each problem is also given. Find that in which order the contestant must solve the problems to maximize the score.

Score $\rightarrow S_1 \quad S_2 \quad S_3 \quad \dots \quad S_n$
 Decay $\rightarrow D_1 \quad D_2 \quad D_3 \quad \dots \quad D_n$
 Time $\rightarrow T_1 \quad T_2 \quad T_3 \quad \dots \quad T_n$

$$(\text{marks gained from one question } (Q)) = S_i - D_i T (\text{elapsed from start})$$

④ Let's say there are two problems

	Q_1	Q_2	} → if I solve Q_1 before Q_2
$S \rightarrow$	5	3	
$D \rightarrow$	2	1	
$T \rightarrow$	1	2	

Net Score = (Score for Q_1) + (Score for Q_2)

→ if I solve Q_2 before Q_1

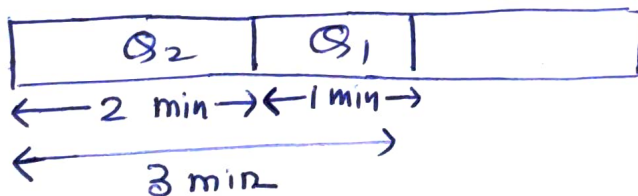
Net Score = (Score for Q_2) + (Score for Q_1)

$= (3 - \underset{\substack{\uparrow \\ \text{Time from start}}}{2} \times 1) + (5 - \underset{\substack{\uparrow \\ \text{Time from start}}}{2} \times \underset{\substack{\uparrow \\ \text{Time from start}}}{3})$

$= 1 + (-1) = 0$

$= (5 - 1 \times 2) + (3 - 1 \times \underset{\substack{\uparrow \\ \text{Time from start}}}{3})$
 $= 3 + 0 = 3$

ordering of problems changes the score



Let's find out the order of sorting by swapping technique:

	Q_1	Q_2
$S \rightarrow$	S_1	S_2
$D \rightarrow$	D_1	D_2
$T \rightarrow$	T_1	T_2

Let's say, optimal ordering is first Q_1 then Q_2

then
 $\text{score}(Q_1 \rightarrow Q_2) \geq \text{score}(Q_2 \rightarrow Q_1)$

Score($Q_1 \rightarrow Q_2$) = $(S_1 - D_1 \times T_1) + (S_2 - D_2(T_1 + T_2))$

Score($Q_2 \rightarrow Q_1$) = $(S_2 - D_2 \times T_2) + (S_1 - D_1(T_1 + T_2))$

$$(s_1 - d_1 \times T_1) + (s_2 - d_2(T_1 + T_2)) \geq (s_2 - d_2 \times T_2) + (s_1 - d_1(T_1 + T_2))$$

$$\Rightarrow -d_2 T_1 \geq -d_1 T_2$$

$$\Rightarrow d_2 T_1 \leq d_1 T_2$$

$$\Rightarrow \left(\frac{T_1}{d_1}\right) \leq \left(\frac{T_2}{d_2}\right)$$

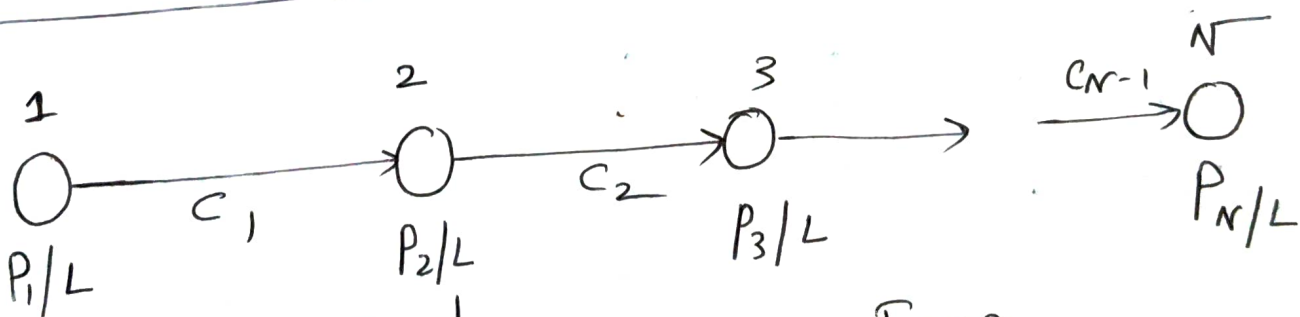
So, your optimal ordering $(Q_1 \rightarrow Q_2)$ is true iff

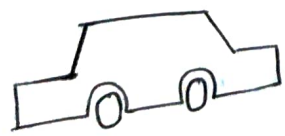
$$\left(\frac{T_1}{d_1}\right) \leq \left(\frac{T_2}{d_2}\right)$$

So, we need to sort solve questions in ascending order of $\left(\frac{T_i}{d_i}\right)$ for maximum score.

When you are able to guess, that there might be an optimal order for this question, then by swapping proof, we can get the order or the sorting criteria.

Q3.



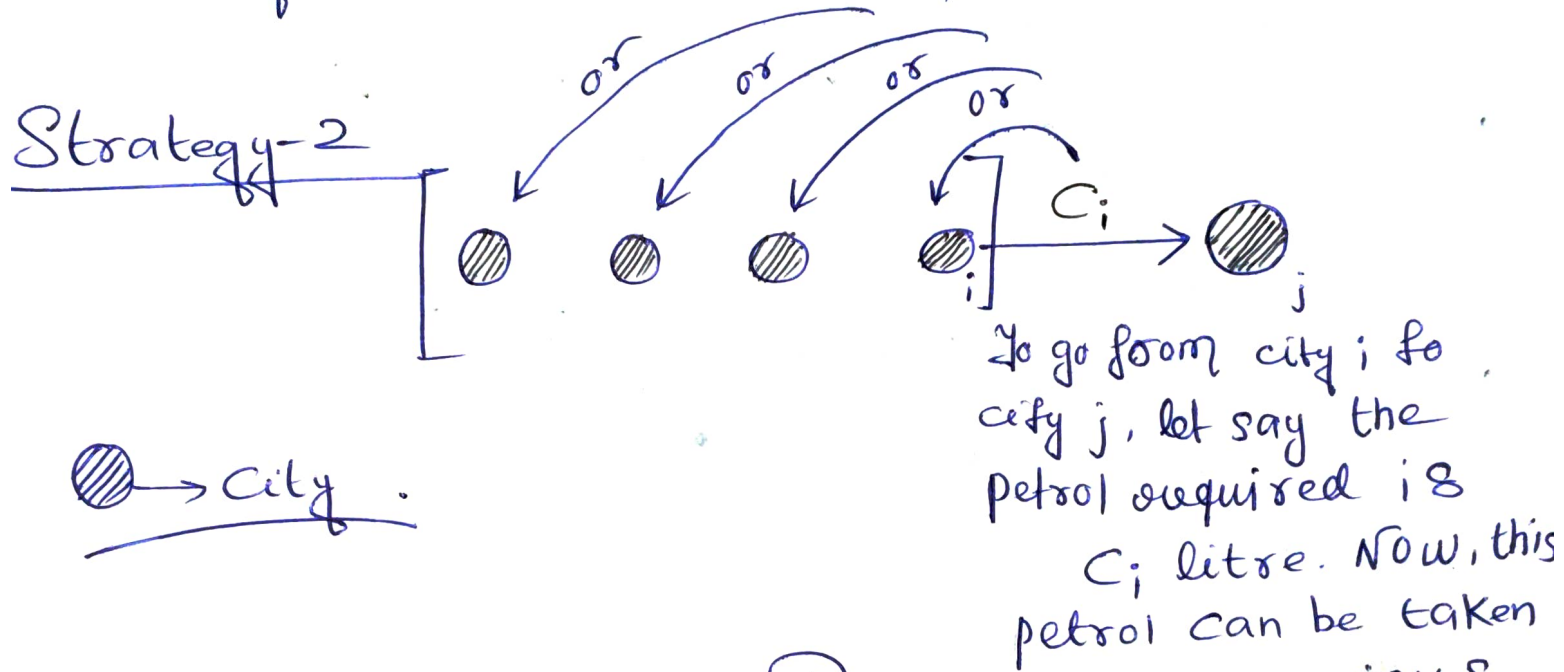
 • Find (Minimum Cost to move From city 1 to city N)

$(P_i \rightarrow$ Cost of petrol per litre at i th city
 $(C_i \rightarrow$ How many L of petrol needed to travel

Initially tank is empty.

At each petrol pump, there is unlimited petrol available, and your tanker has unlimited capacity.

Strategy 1 $\rightarrow$ Fill that much petrol, that would take you to the next cheaper petrol pump.

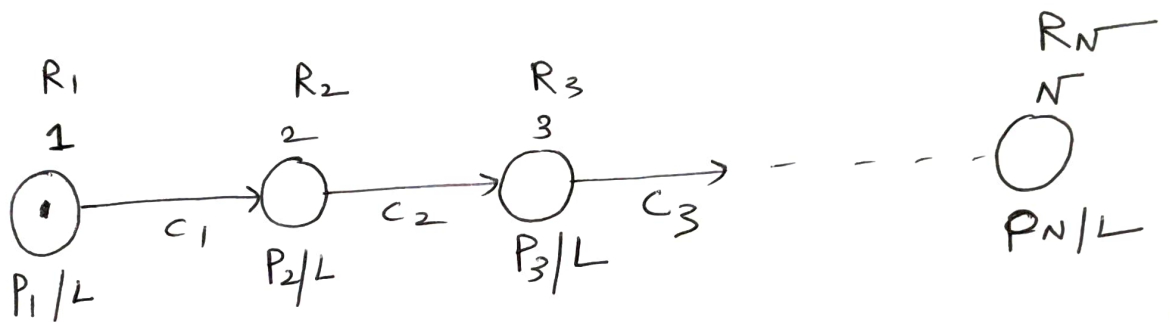


From any of the previous cities. So, greedily for the minimum cost, I will take From the petrol pump which has minimum petrol cost (From the previous cities). So, simply maintain a prefix minimum and multiply with cost at each city.

$$\sum C_i [\min(P_i) \text{ till now}]$$

Now, little modification to the previous problem. Now, each petrol pump don't have unlimited petrol. there is a max capacity of petrol that you can fill at each petrol pump.

Let say,
 $R_i \rightarrow$ Max. L of petrol you can fill at the i^{th} pump.



So, here also idea is same, but there is a associated ^{pump} maximum petrol, which is associated to each petrol pump, so after petrol finished in that minimum cost pump, we will look for next minimum pump, among the previous. So, for this we need a data structure that will have all the petrol pumps, along with their per litre cost and remaining capacity. So, for that, we shall use a priority queue which will hold all the previous pumps with their respective (P_i, R_i)